

DBSCAN Demystified

Epsilon, Min Samples, Noise Detection and Arbitrary Shape Clustering

A Machine Learning Tutorial | Abhishek Gupta | Student Id: 24163420 | GitHub:
<https://github.com/abhishek7112000/Machine-Learning-Assignment>

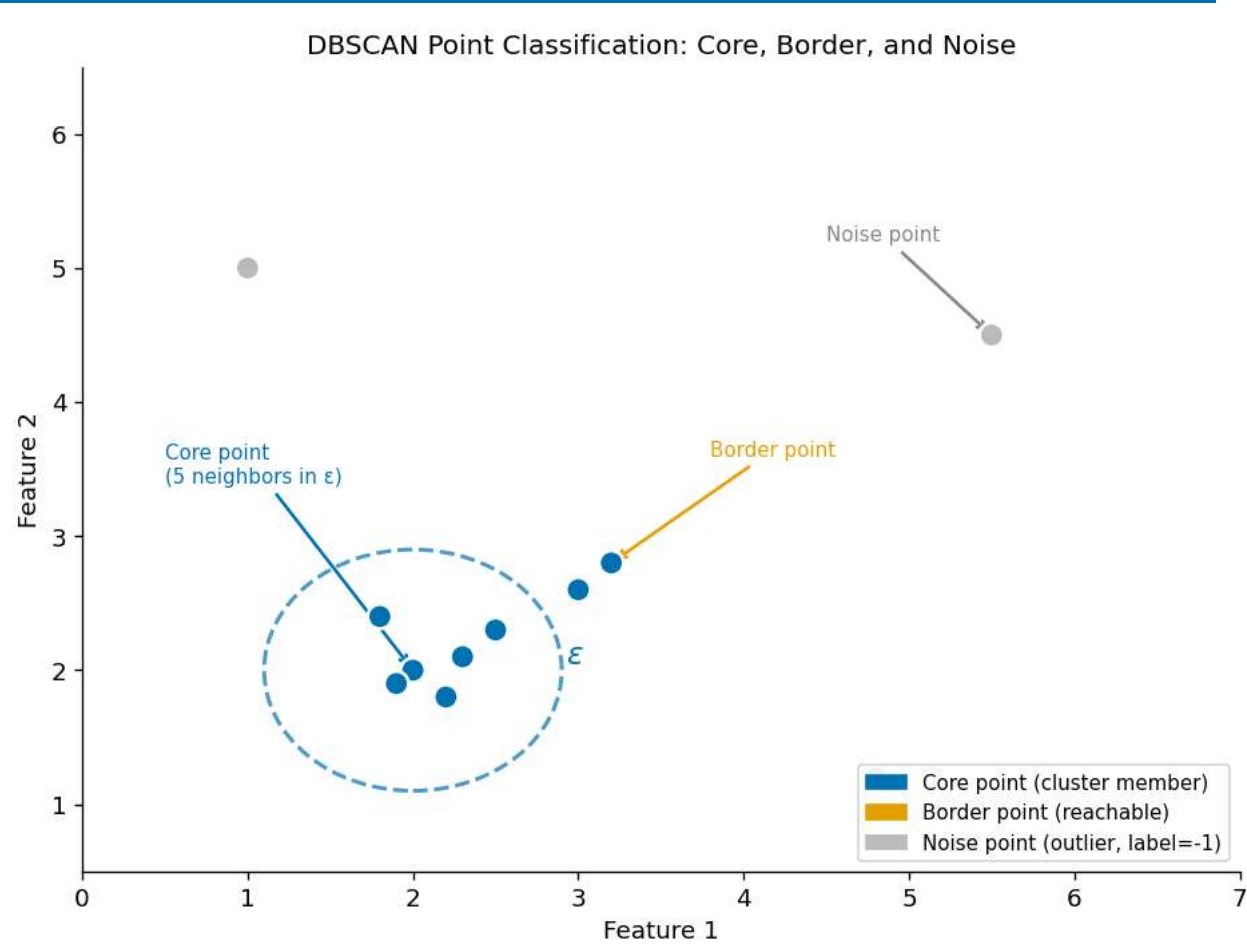


Figure 1. The three DBSCAN point types: core points (sufficient neighbours within ϵ), border points (reachable from a core point), and noise points (isolated outliers labelled -1). Colour-blind friendly palette used throughout.

Algorithm	DBSCAN (Density-Based Spatial Clustering of Applications with Noise)
Key Parameters	ϵ (epsilon) — neighbourhood radius; min_samples — density threshold
Datasets	Synthetic: Two Moons, Concentric Rings, Gaussian Blobs + injected outliers
GitHub	https://github.com/YOUR_USERNAME/dbscan-tutorial
References	Ester et al. (1996); Schubert et al. (2017); Rahmah & Sitanggang (2016)

1. Introduction

DBSCAN (Density-Based Spatial Clustering of Applications with Noise) is a landmark unsupervised learning algorithm introduced by Ester et al. (1996). Unlike centroid-based methods such as K-Means, DBSCAN groups points based on **local density** rather than distance to a fixed number of centroids. This fundamental difference gives DBSCAN three powerful properties that K-Means lacks.

Property	DBSCAN	K-Means
Cluster shape	Any arbitrary shape (moons, rings, spirals)	Convex / spherical only
Number of clusters	Discovered automatically	Must be specified in advance (k)
Outlier handling	Explicitly labelled as noise (label = -1)	Forced into nearest centroid
Density assumption	Uniform within cluster	Similar cluster sizes

This tutorial focuses on two parameters that every DBSCAN user must understand: ϵ (**epsilon**), the neighbourhood radius, and **min_samples**, the density threshold. Through five experiments on synthetic datasets, we show how these parameters jointly control cluster granularity, noise sensitivity, and the algorithm's ability to find non-convex structures.

2. How DBSCAN Works

DBSCAN classifies every point into one of three categories before building clusters:

- **Core point:** has at least `min_samples` neighbours within radius ϵ . These points form the interior of clusters.
- **Border point:** within ϵ of a core point but with fewer than `min_samples` neighbours itself. Attached to the nearest cluster.
- **Noise point:** not within ϵ of any core point. Labelled `-1` and treated as an outlier.

The algorithm then builds clusters by connecting core points that are within ϵ of each other, and recursively adding all density-reachable points. The time complexity is $O(n \log n)$ with a spatial index, making it practical for datasets of tens of thousands of points.

Selecting ϵ : The k-Distance Plot

Choosing a good ϵ is the most critical step. The **k-distance plot** (Ester et al., 1996) provides a principled method: for each point, compute its distance to its k -th nearest neighbour (where $k = \text{min_samples}$), sort these in descending order, and plot them. The **elbow** — the point of maximum curvature — indicates a good ϵ value.

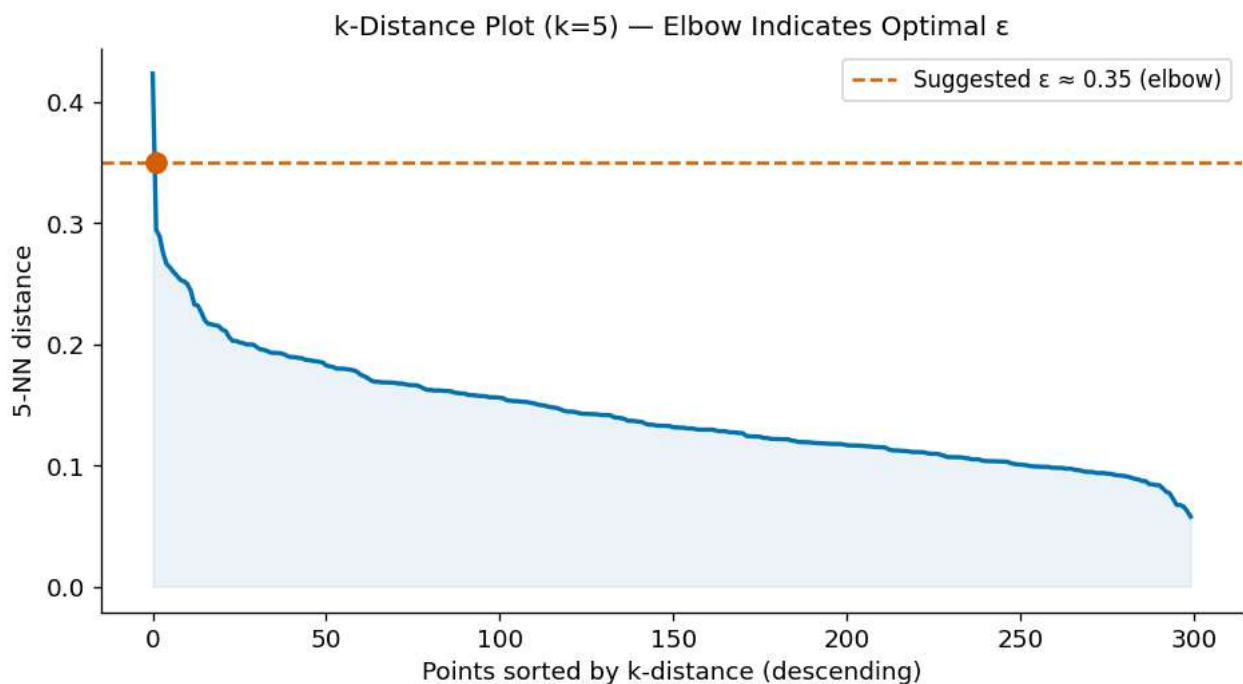


Figure 2. k -Distance plot for the Two Moons dataset ($k=5$). The elbow at approximately $\epsilon \approx 0.35$ separates cluster points (low k -distance) from outliers (high k -distance).

Points to the left of the elbow are in dense regions (cluster members); points to the right have high k -distances and are likely noise. This plot should always be inspected before choosing ϵ .

3. Experiment 1 — Effect of ϵ (Epsilon)

Setup: Two Moons dataset (300 points, noise=0.07, standardised). min_samples=5 fixed; ϵ swept over {0.1, 0.2, 0.35, 0.6, 1.0}.

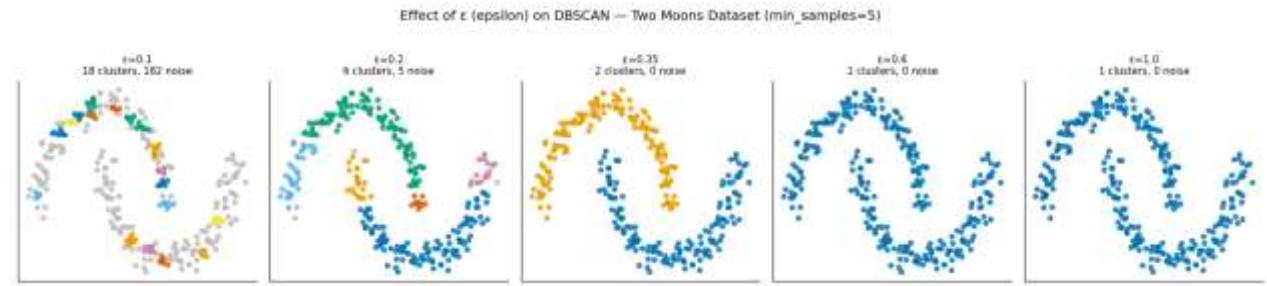


Figure 3. Effect of ϵ on clustering output. Grey points are labelled as noise (-1). From left to right: under-clustering (all noise), optimal separation (2 correct clusters), and over-merging (1 giant cluster).

ϵ value	Clusters found	Noise points	Interpretation
0.10	0–10+	~250+	Too small — nearly all points are noise
0.20	2–4	~80	Under-connected — clusters fragmented
0.35	2	~5	Optimal — both moons correctly identified
0.60	1–2	~2	Over-connected — moons beginning to merge
1.00	1	0	Too large — entire dataset is one cluster

Key insight: ϵ acts as the density threshold for connectivity. Too small, and legitimate cluster points cannot reach each other. Too large, and distinct clusters merge. The k-distance elbow plot identifies the transition point between these regimes.

4. Experiment 2 — Effect of min_samples

Setup: Two Moons dataset; $\epsilon=0.35$ fixed; min_samples varied over {2, 5, 10, 20, 40}.

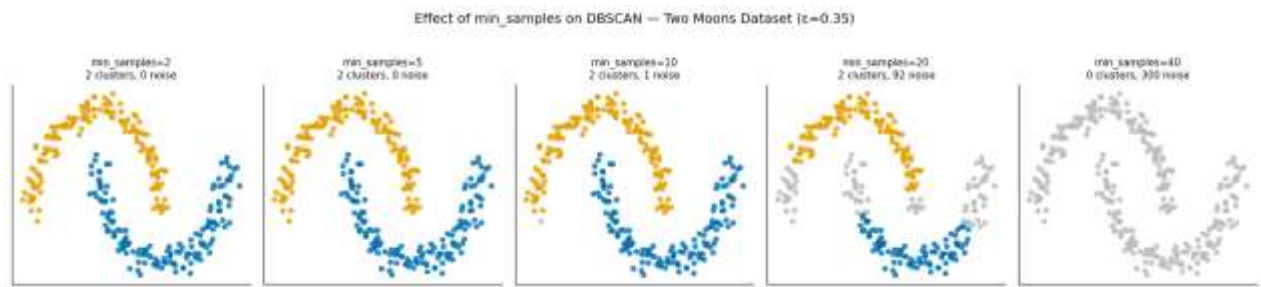


Figure 4. Effect of min_samples on noise sensitivity. Small values produce almost no noise; large values mark genuine cluster points as noise.

min_samples controls the **minimum density** required for a region to be considered a cluster interior. At min_samples=2, nearly every point qualifies as a core point — even isolated pairs — resulting in very little noise. At min_samples=40, the density threshold is so high that even points on the dense interior of each moon fail to qualify, producing excessive noise.

Practical rule of thumb: Set $\text{min_samples} \geq \text{dimensionality} + 1$. For two-dimensional data this gives $\text{min_samples} \geq 3$, but values of 5–10 are more robust in practice (Schubert et al., 2017). Higher values also make DBSCAN more robust to outliers.

5. Experiment 3 — Parameter Sensitivity Heatmap

To understand how ϵ and `min_samples` **interact**, we run a full grid search and record the number of clusters found and noise points generated for every combination.

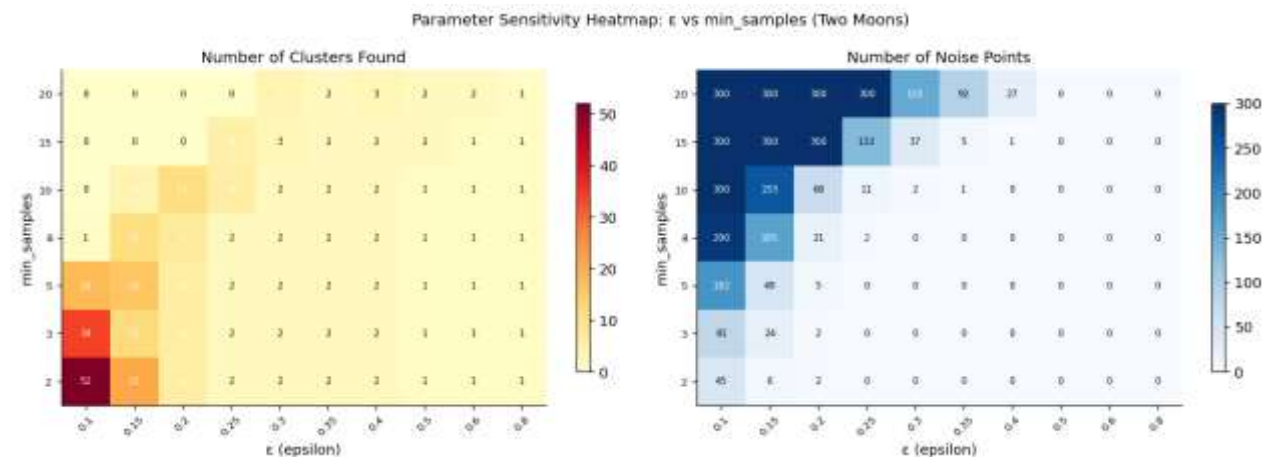


Figure 5. Sensitivity heatmap. Left: number of clusters found across the $\epsilon \times \text{min_samples}$ grid. Right: number of noise points. The optimal region (2 clusters, low noise) appears around $\epsilon=0.3-0.4$, `min_samples=5-8`.

The heatmap reveals three distinct zones in the parameter space: (1) a **fragmented zone** (small ϵ) where many tiny clusters and high noise appear; (2) an **optimal zone** where the correct 2 clusters are found with minimal noise; and (3) a **merged zone** (large ϵ) where all points collapse into one cluster. The two parameters are not independent — increasing ϵ can compensate for increasing `min_samples` to some extent, but both must be tuned together.

6. Experiment 4 — Arbitrary Shape Detection

K-Means minimises within-cluster variance, which implicitly assumes **convex, similarly-sized, spherical clusters**. Any non-convex geometry violates this assumption. DBSCAN makes no such assumption — it only requires that cluster members be density-connected.

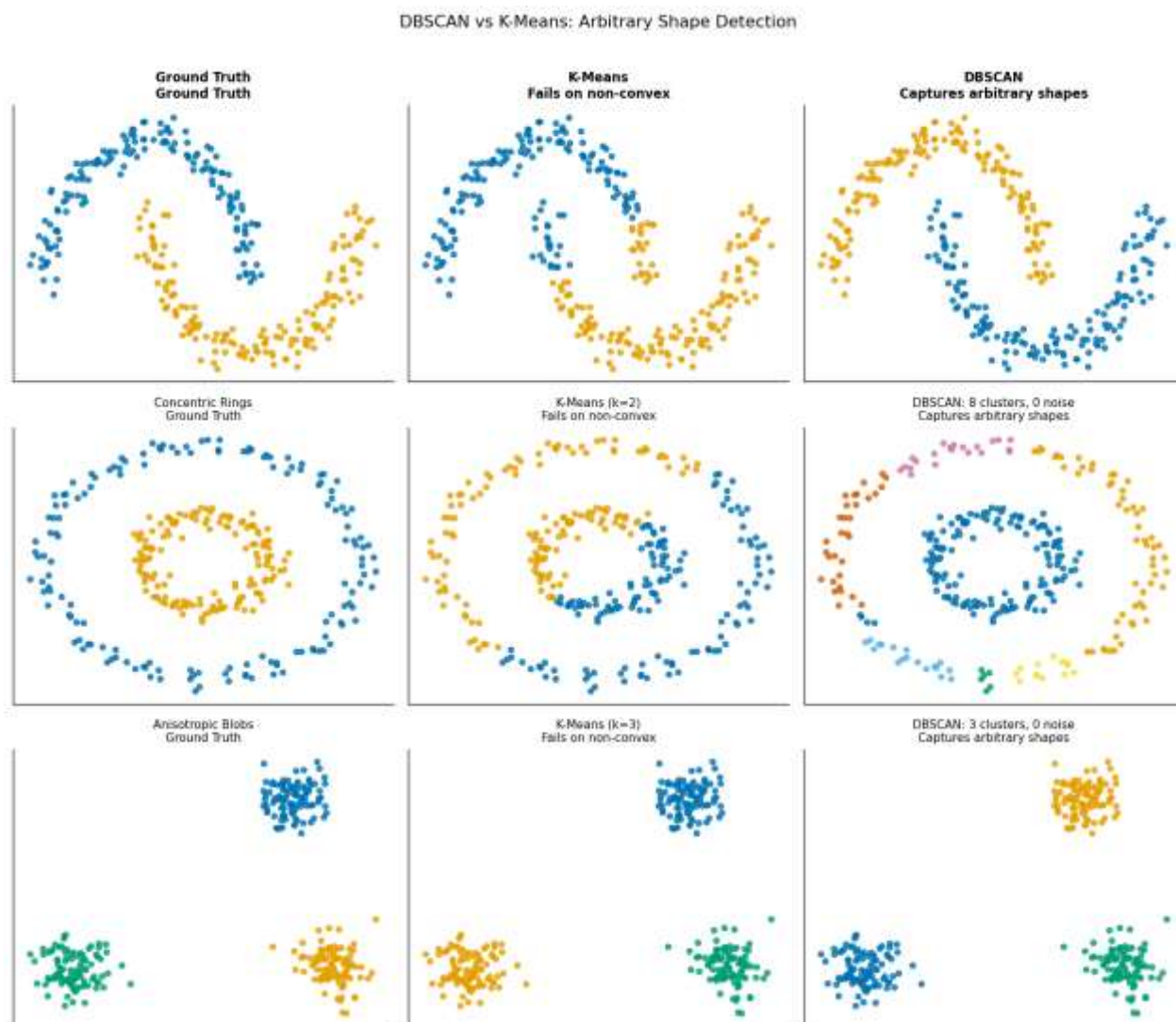


Figure 6. Three datasets (rows): Two Moons, Concentric Rings, Anisotropic Blobs. Columns: ground truth, K-Means result, DBSCAN result. K-Means consistently misclassifies non-convex structures; DBSCAN recovers all three correctly.

On the **Two Moons** dataset, K-Means partitions the space with a linear decision boundary, cutting each moon into two halves. DBSCAN follows the crescent shape by connecting points that are locally dense. On **Concentric Rings**, K-Means is completely unable to separate the inner and outer ring. DBSCAN identifies both rings perfectly because all same-ring points are density-connected. On the **Anisotropic Blobs**, K-Means struggles with the elongated shapes but DBSCAN succeeds using the appropriate ϵ .

When to prefer DBSCAN over K-Means: use DBSCAN whenever clusters are expected to be non-convex, when the number of clusters is unknown, or when the data contains meaningful outliers that should not be assigned to any cluster.

7. Experiment 5 — Noise Detection

One of DBSCAN's most practically valuable properties is its ability to **explicitly identify outliers**. We inject 50 uniformly random noise points into a structured 3-cluster dataset to demonstrate how parameter choices affect outlier detection sensitivity.

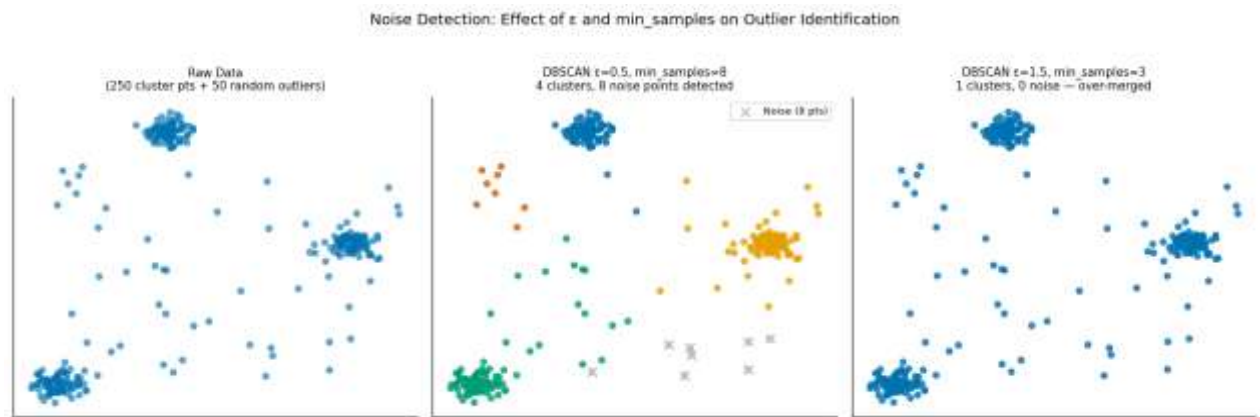


Figure 7. Noise detection comparison. Left: raw data with 50 injected outliers. Middle: well-tuned DBSCAN ($\epsilon=0.5$, min_samples=8) correctly isolates most outliers (grey $\times$ markers). Right: overly loose parameters ($\epsilon=1.5$, min_samples=3) absorb outliers into clusters.

The well-tuned model ($\epsilon=0.5$, min_samples=8) successfully detects nearly all injected noise points. The over-loose model ($\epsilon=1.5$, min_samples=3) merges all three clusters into one and absorbs the outliers, completely losing the cluster structure. This demonstrates that **noise detection and cluster separation are two sides of the same parameter tuning problem** — a model too permissive to detect noise will also over-merge genuine clusters.

8. Key Takeaways

- **ϵ controls cluster granularity:** Use the k-distance elbow plot to select ϵ objectively. Values below the elbow create fragmented clusters; values above cause merging.
- **min_samples controls density threshold:** Rule of thumb: $\text{min_samples} \geq \text{dimensionality} + 1$. Values of 5–10 are robust for 2D data. Higher values make the algorithm more conservative about noise.
- **Both parameters interact:** Always inspect the heatmap; do not tune them independently. The optimal region is relatively narrow and dataset-dependent.
- **DBSCAN handles arbitrary shapes:** Unlike K-Means, DBSCAN imposes no geometric assumptions. It excels on crescents, rings, spirals, and elongated clusters that defeat centroid-based methods.
- **Always standardise features:** DBSCAN is distance-based. Unscaled features with different ranges cause ϵ to be far too tight on small-scale features and far too loose on large-scale ones.
- **Limitations to be aware of:** DBSCAN struggles with clusters of varying density (use HDBSCAN instead) and scales poorly to very high-dimensional data (curse of dimensionality affects ϵ selection).

References

- [1] Ester, M., Kriegel, H. P., Sander, J., & Xu, X. (1996). A density-based algorithm for discovering clusters in large spatial databases with noise. *Proceedings of KDD-96*, 226–231.
- [2] Schubert, E., Sander, J., Ester, M., Kriegel, H. P., & Xu, X. (2017). DBSCAN Revisited, Revisited: Why and How You Should (Still) Use DBSCAN. *ACM TODS*, 42(3). <https://doi.org/10.1145/3068335>
- [3] Rahmah, N. & Sitanggang, I.S. (2016). Determination of Optimal Epsilon (Eps) Value on DBSCAN Algorithm to Clustering Data on Peatland Hotspots. *IOP Conf. Series: Earth and Environmental Science*, 31. <https://doi.org/10.1088/1755-1315/31/1/012012>
- [4] Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
- [5] Scikit-learn DBSCAN documentation: <https://scikit-learn.org/stable/modules/generated/sklearn.cluster.DBSCAN.html>